ANNS SIMD 实验 — 上下文信息

系统环境

开发 & 测试环境矩阵

环境	SIMD	能开发?	能编译跑?	硬件 perf/VTune?	角色
Windows 11 本机（i9-13900H）	AVX2 + VNNI	✅	✅（写 x86 SIMD 版本后可编译跑，无 ARM intrinsics不影响 x86 代码）	✅ **唯一能做 VTune/perf 的环境**	**下一步主力**：写 AVX2 版本 + VTune 采集
鲲鹏 920（服务器）	NEON	❌（只能 `test.sh` 提交）	✅	❌（log 权限被限）	ARM 数据采集（已完成）
3090 Docker（Xeon 8358）	AVX-512	✅（支持 agent）	✅	❌（容器无 CAP_PERFMON，perf_event_paranoid=4）	**降为可选**：只能多拿一组 AVX-512 速度数据，perf 做不了，ROI 不高
~~平板 Termux~~	NEON asimddp	❌已弃	❌已弃	❌（PRoot 无 PMU）	历史数据保留

⚠️ **3090 环境限制确认**（agent 归纳）：硬件是 Intel Xeon Platinum 8358，支持 arch_perfmon / pebs / intel_pt，**硬件没问题**。但当前是 Docker 容器，`perf_event_paranoid=4` 且缺 `CAP_PERFMON/SYS_ADMIN/SYS_PTRACE`，`perf stat` 直接报权限错误；也没装 VTune 驱动。结果：**3090 只能跑汇编/速度对比，不能跑硬件 PMU 采样**。除非学长配合放宽容器权限。

主开发环境（原生 Windows 11）

✨ **2026-04-17 更新**：平板 Termux 已弃用。日常代码开发在**原生 Windows 11**上（不用 WSL），鲲鹏服务器跑 ARM 数据，3090 服务器跑 x86 汇编/速度对比。**硬件 perf/VTune 采集只能在 Windows 本机做**（3090 是 Docker 容器无 PMU 权限，鲲鹏/平板同样受限）。

项目	详情
设备	Windows 11 笔记本（Intel i9-13900H, 10 物理核 20 线程）
系统	Windows 11（**未使用 WSL**，直接用 PowerShell / Git Bash）
CPU 特性	avx, avx2, avx_vnni, sse4.2, fma, aes —— 注意**无 AVX-512**（3090 服务器才有）
编译器	不在本地编译（Windows 没有 `arm_neon.h`），仅做代码编辑 + git
工作目录	`D:\Study\26sp\parallel\ann\`
用途	整理代码、提交 git、整理上下文。**本地不跑实验**（所以没有 `files/` 目录）

ℹ️ 注：上次 agent 扫描运行时输出的 `uname -a` 显示 WSL2 内核，那只是 agent 执行 shell 命令时附带用了一下 WSL。代码仓库本身放在 `D:\` 盘，用户日常用 Windows 原生环境（VSCode + Git Bash / PowerShell）。

~~平板 Termux~~（已弃用，只留下历史数据）

远程服务器（鲲鹏 920）

项目	详情
平台	ARM 鲲鹏 920 服务器
SIMD	NEON 指令集
跳板机	`s2413575@10.137.144.91:9001`

项目	详情
目标服务器	`s2413575@192.168.90.141`
登录命令	`ssh -J s2413575@10.137.144.91:9001 s2413575@192.168.90.141`

📁 文件目录

服务器目录结构（鲲鹏 920）

```
/home/s2413575/ann/
├── files/              # 空目录（数据在 /anndata/）
├── hnswlib/            # hnswlib 库
├── flat_scan.h
├── flat_scan_simd.h
├── sq_scan_simd.h
├── pq_scan_simd.h
├── main.cc             # 当前为 PQ-SIMD, data_path="/anndata/"
├── main_baseline.cc    # 4 个版本可轮换
├── main_flatsimd.cc
├── main_sqsimd.cc
├── main_pqsimd.cc
├── run_all.sh          # 依次跑 4 版本的脚本
├── qsub.sh
└── test.sh             # 提交用（加密，不可改）
```

公共数据目录

```
/anndata/
├── DEEP100K.base.100k.fbin      # 37MB — 100K 条 96维 float32 向量
├── DEEP100K.query.fbin           # 3.7MB — 10K 条查询向量
└── DEEP100K.gt.query.100k.top100.bin  # 7.7MB — groundtruth top100
```

Windows 本地目录结构（`D:\Study\26sp\parallel\ann\`）

```
ann/
├── main.cc                   # PQ-SIMD 版本（data_path="/anndata/"）
├── main_baseline.cc          # 4 版本切换用
├── main_flatsimd.cc
├── main_sqsimd.cc
├── main_pqsimd.cc
├── flat_scan.h / flat_scan_simd.h / sq_scan_simd.h / pq_scan_simd.h
├── hnswlib/                  # HNSW 库
├── hnswlib.zip
├── qsub.sh / test.sh
├── run_all.sh                # 依次跑 4 版本的脚本
├── README.md                 # GitHub 仓库的 README
├── result_baseline.txt       # 鲲鹏跑出来的 4 份原始结果
├── result_flatsimd.txt
├── result_sqsimd.txt
├── result_pqsimd.txt
├── test.e / test.o / main    # 上次从鲲鹏拉回来的产物（可删）
├── analysis/                 # 汇编分析代码
│   ├── analyze.cc
│   ├── bench_optlevel.cc
│   ├── run_asm_analysis.sh
│   └── run_simpleperf_analysis.sh
├── benchmarks/               # E1/E2/E3 benchmark 代码
```

```
│   ├── bench_experiments.cc
│   ├── plot_tradeoff.py
│   └── run_android_termux_proot.sh
├── bench_results/            # 分平台的实验结果
│   ├── android_termux_proot_ubuntu/  # ✅ 完整 CSV+PNG+RESULTS.md
│   ├── kunpeng_server/               # ⚠ 只有 README，待把 result_*.txt 整理进来
│   └── windows/                      # ⚠ 只有 README（开发机不跑数据，可以删）
└── local_results/
    └── termux_arm64/         # 历史平板汇编分析完整产物（.s / objdump / neon_stats / simplepe
rf 等）
```

✨ **路径统一**：所有 4 个 main*.cc 都已统一用 `data_path = "/anndata/"`，与鲲鹏服务器一致。本地 Windows 不跑测试，不需要 `./files/`。

🔗 Git 仓库

- **地址**：https://github.com/kyc001/parallel
- **分支**：`master`
- **结构**：`parallel/ann/` 下存放所有 ANN 相关代码
- **认证**：SSH 方式（`git@github.com:kyc001/parallel.git`），非 PAT
- **注意**：`.gitignore` 已排除 `ann/files/`（数据文件）和 `ann/main`（编译产物）
- **当前状态**：工作区干净，最新 commit `ef7472f: feat(windows-avx2): add i9-13900H AVX2 impl + VTune profiling`（Day 13 归档，48 files changed, +3234/-1）；上一个 commit `ee3e7df: chore: organize Kunpeng results, update README, cleanup`

📊 实验数据

核心对比（Baseline → Flat-SIMD → SQ-SIMD）

版本	Recall@10	Latency (μs)	vs Baseline 加速比
串行 Flat（无 SIMD）	0.99995	8282.96	1.00x
Flat-SIMD（float32 NEON 4路展开）	0.99995	2596.54	**3.19x**
SQ-SIMD（p=1000）	0.99995	1875.51	**4.42x**
SQ-SIMD（p=100，最快）	0.99995	1087.87	**7.61x**

SQ-SIMD rerank_p 参数调优（Latency-Recall Tradeoff）

rerank_p	Recall@10	Latency (μs)	vs Baseline 加速比
100	0.99995	1087.87	7.61x
200	0.99995	1387.96	5.97x
500	0.99995	1856.89	4.46x
1000	0.99995	2399.20	3.45x
2000	0.99995	3449.92	2.40x
5000	0.99995	5649.49	1.47x
10000	0.99995	10293.17	0.80x
50000	0.99995	30101.22	0.28x
100000（全量）	0.99995	45187.62	0.18x

💡 **关键发现：**

- Recall 在所有 p 值下都保持 0.99995，说明 SQ 量化精度非常高，即使只 rerank 100 个候选也能找到正确结果
- SQ 的量化误差极小，uint8 L2 距离的排序与 float32 IP 距离高度一致
- **最佳配置 p=100**：latency 仅 1088μs，相比串行 baseline 加速 **7.61 倍**，recall 无损
- p ≥ 10000 时反而比串行 Flat 更慢，因为 uint8 粗排遍历 + float32 精排的总开销超过了纯 float32 遍历
- Latency 与 p 近似线性关系，符合 O(N·d_uint8 + p·d_float32) 的复杂度分析

PQ-SIMD rerank_p 参数调优（M=8, dsub=12）

rerank_p	Recall@10	Latency (μs)	vs Baseline 加速比
100	0.70930	842.05	9.83x
200	0.83835	924.83	8.96x
500	0.94740	1178.43	7.03x
1000	0.98355	1520.52	5.45x
2000	0.99550	2488.09	3.33x
5000	0.99980	4180.43	1.98x
10000	0.99995	7268.55	1.14x
50000	0.99995	22651.73	0.37x
100000（全量）	0.99995	34630.02	0.24x

💡 **PQ vs SQ 对比分析：**

- PQ 粗排更快：ADC 每条向量只需 8 次查表累加，远快于 SQ 的 96 维 uint8 计算
- PQ 量化误差更大：小 p 值时 recall 明显下降（p=100 时只有 0.709），而 SQ 在所有 p 值下 recall 都是 0.99995
- **PQ 优势区间**：p=1000-2000 时，recall ≥ 0.98 且 latency 明显低于 SQ
- **SQ 优势区间**：需要极高 recall（0.999+）时，SQ 可以用很小的 p 就达到
- **内存压缩**：PQ 48x 压缩 vs SQ 4x 压缩，PQ 在大规模数据上内存优势巨大

🖥️ 鲲鹏 920 服务器完整数据（2026-04-17 四版本全量补测）

(1) 核心方法对比（N=100K, k=10, 2000 queries）

版本	Recall	鲲鹏 (μs)	平板 (μs)	平板/鲲鹏	鲲鹏内部加速 vs 自身 Baseline
串行 Flat (Baseline)	0.99995	**16120.74**	8282.96	平板 1.95x	1.00x
Flat-SIMD	0.99995	**5821.16**	2596.54	平板 2.24x	**2.77x**
SQ-SIMD (p=100)	0.99995	**2422.68**	1087.87	平板 2.23x	**6.65x**
PQ-SIMD (p=1000)	0.98355	**1984.35**	1520.52	平板 1.30x	**8.12x**

(2) SQ-SIMD 9 p 值（鲲鹏 vs 平板）

p	Recall	鲲鹏 (μs)	平板 (μs)	鲲鹏加速 vs 自身 Baseline	100	0.99995	2422.68
200	0.99995	2422.90	1387.96	6.65x	500	0.99995	2701.90
1000	0.99995	2997.13	2399.20	5.38x	2000	0.99995	3674.23
5000	0.99995	5391.92	5649.49	2.99x	10000	0.99995	8033.46
50000	0.99995	28138.01	30101.22	0.57x	100000	0.99995	40260.6

(3) PQ-SIMD 9 p 值（鲲鹏 vs 平板，M=8, dsub=12）

p	Recall	鲲鹏 (μs)	平板 (μs)	鲲鹏加速 vs 自身 Baseline	100	0.70930	1224.20
200	0.83835	1315.36	924.83	12.26x	500	0.94740	1566.47
1000	0.98355	1984.35	1520.52	8.12x	2000	0.99550	2693.58
5000	0.99980	4506.40	4180.43	3.58x	10000	0.99995	7586.31
50000	0.99995	23857.74	22651.73	0.68x	100000	0.99995	42163.4

🔍 **跨 ARM 平台发现（四版本完整对比）：**

1. **Recall 完全一致**：两平台所有配置下 recall 数值一模一样，证明算法在不同 ARM 微架构上的结果确定性
2. **鲲鹏自身加速比更显著**：鲲鹏 Baseline 基线更慢（16120μs vs 平板 8282μs），所以 SQ/PQ 相对 Baseline 的加速更大——**PQ p=100 上鲲鹏获得 13.17x 加速，平板只有 9.83x**
3. **绝对延迟上平板仍领先**：大多数配置下平板比鲲鹏快 1.30x–2.24x，尤其 Flat 类（2.24x）差距最大
4. **可能解释**：
   - 平板是较新的 Cortex-A 核心（asimddp），单核主频很可能比鲲鹏 920 的 2.6GHz 更高
   - 鲲鹏 920 是服务器 CPU，优化方向为多核吞吐而非单线程延迟
   - 平板可能处于短时性能模式
5. **报告价值**：既证明了算法的跨平台正确性，也为"同架构跨微架构性能分析"提供了真实数据素材

测试环境说明

- **数据集**：DEEP100K（96维，100K 条 float32 向量，IP 距离）
- **查询数**：前 2000 条
- **k**：10
- **主测试平台**：平板 Termux（ARM 8核，NEON asimd + asimddp）
- **补测平台**：鲲鹏 920 服务器（ARM AArch64，通过 test.sh 提交）

🔍 汇编分析与性能剖析

NEON 指令统计（不同优化等级）

优化等级	汇编行数	NEON float ops	NEON load/store	NEON integer ops	scalar float ops
-O0	673	23	116	6	6
-O1	299	25	6	6	6
-O2	329	34	10	6	9
-O3	451	56	28	36	22

编译器自动向量化分析

📝 **关键发现：**

- **-O2**：串行 `ip_distance_serial` **未被自动向量化**，仍用标量 `fmadd s`。编译器报告：`couldn't vectorize loop`
- **-O3**：编译器对串行版本做了自动向量化（`loop vectorized using 16 byte vectors`），但规约部分仍逐 lane 累加，不如手写 SIMD 的多累加器结构规整
- 手写 NEON intrinsic 不会被编译器"再次向量化"，而是直接映射为 NEON 指令

关键函数汇编摘要

ip_distance_simd (-O2)：4 路累加器，每轮 16 个 float

```
ldp  q16, q6, [x3]         // 一次加载 8 个 float
ldp  q7, q5, [x4]
ldp  q4, q2, [x3, 32]
ldp  q3, q1, [x4, 32]
fmul v7.4s, v16.4s, v7.4s  // 4 路并行乘法
fmul v5.4s, v6.4s, v5.4s
fmul v3.4s, v4.4s, v3.4s
fmul v1.4s, v2.4s, v1.4s
fadd v28.4s, v28.4s, v7.4s // 4 个独立累加器
fadd v29.4s, v29.4s, v5.4s
fadd v0.4s, v0.4s, v3.4s
fadd v31.4s, v31.4s, v1.4s
```

sq_l2_distance_simd (-O2)：uint8 NEON 距离

```
ldr   q0, [x0, x3]           // 加载 16 个 uint8
ldr   q28, [x1, x3]
uabd  v28.16b, v0.16b, v28.16b  // 绝对差
umull  v29.8h, v28.8b, v28.8b    // 平方 (low)
umull2 v28.8h, v28.16b, v28.16b  // 平方 (high)
uadalp v30.4s, v29.8h            // 累加
uadalp v30.4s, v28.8h
addv   s31, v30.4s               // 水平归约
```

adc_distance (-O2)：标量查表累加（无法 SIMD 化，因为间接索引 gather 访问）

```
ldrb w4, [x1, x3]              // 读取 code[m]
add  w4, w4, w3, lsl 8         // m * 256 + code[m]
ldr  s1, [x0, x4, lsl 2]       // 查 LUT
fadd s31, s31, s1              // 标量累加
```

不同优化等级性能对比（单次调用 96 维 IP 距离）

优化等级	Serial (ns/call)	SIMD (ns/call)	SIMD 加速比
-O0	468.62	269.90	1.74x
-O1	120.81	31.65	3.82x
-O2	90.85	29.53	**3.08x**
-O3	96.92	26.19	3.70x

💡 **分析**：

- -O0 下 SIMD 加速仅 1.74x，因为编译器没有优化寄存器分配和内存访问，NEON 指令的优势被大量 load/store 抵消
- -O1 到 -O2 串行版本从 120.81ns 降到 90.85ns，但仍未自动向量化
- -O3 串行版本被自动向量化后（96.92ns）仍然比手写 SIMD（26.19ns）慢 3.70x，说明手写 SIMD 的多累加器结构的流水线利用率明显优于编译器自动向量化
- 即使在 -O3 下，手写 SIMD 仍然占优，证明手动优化的必要性

🖥️ Windows i9-13900H AVX2 速度数据（2026-04-17）

(1) 核心方法对比（N=100K, k=10, 2000 queries）

版本	Recall	i9-13900H AVX2 (μs)	鲲鹏 NEON (μs)	平板 NEON (μs)	i9 vs 自身 Baseline	i9 vs 鲲鹏	i9 vs 平板
串行 Flat (Baseline)	0.99995	**4571.62**	16120.74	8282.96	1.00x	3.53x	1.81x
SQ-SIMD (p=100)	0.99995	**427.01**	2422.68	1087.87	**10.71x**	5.67x	2.55x

(2) SQ-AVX2 9 p 值完整调优

p	Recall	i9 AVX2 (μs)	鲲鹏 NEON	平板 NEON	vs i9 Baseline	100	0.99995
200	0.99995	469.31	2422.90	1387.96	9.74x	500	0.99995
1000	0.99995	808.63	2997.13	2399.20	5.65x	2000	0.99995
5000	0.99995	2368.87	5391.92	5649.49	1.93x	10000	0.99995
50000	0.99995	11572.16	28138.01	30101.22	0.40x	100000	0.99995

(3) PQ-AVX2 9 p 值完整调优（M=8, dsub=12）

p	Recall	i9 AVX2 (μs)	鲲鹏 NEON	平板 NEON	vs i9 Baseline	100	0.70860
200	0.83360	665.06	1315.36	924.83	6.87x	500	0.94595
1000	0.98330	761.99	1984.35	1520.52	6.00x	2000	0.99550
5000	0.99965	3042.08	4506.40	4180.43	1.50x	10000	0.99995
50000	0.99995	13508.39	23857.74	22651.73	0.34x	100000	0.99995

🚀 **跨三平台关键发现：**

- **i9 AVX2 相对鲲鹏 NEON 全面领先**：Flat 3.31×、SQ@100 5.67×、PQ@1000 2.60×——256-bit 宽度 + Raptor Lake 更高的 IPC + 3 GHz 主频联合优势
- **i9 AVX2 相对平板 NEON 领先 1.5–2.5×**：验证了 128-bit NEON → 256-bit AVX2 硬件宽度翻倍带来的实际收益，非线性扩展（理论 2×，实际 1.5–2.5×，一部分被带宽拖住，与 VTune 的 DRAM Bound 44% 一致）
- **SQ@100 在 i9 达到 10.71× 加速**：仅 427 μs、recall 无损，是三平台中最高的单线程加速比
- **PQ@100 达到 398 μs**（三平台最快绝对延迟），但 recall 0.709 过低；推荐生产配置 PQ@1000 或 SQ@100
- **大 p 区间（p≥50000）平台差距缩小**：i9/鲲鹏比由 Flat 的 3.31× 降到 PQ@100K 的 2.65×——因为大 p 的短板是内存带宽而非计算
- **鲲鹏/平板 PQ@1000 ratio 1.30× → i9/平板 2.00×**：证明 AVX2 对 PQ 查表类工作负载也有显著提升（即使核心 ADC 循环是标量）

🖥️ Windows i9-13900H AVX2 VTune 采集（2026-04-17）

🆕 **采集环境**：Intel i9-13900H (Raptorlake-P)，Windows 11，管理员 PowerShell，MinGW g++ `-O2 -mavx2 -mfma`，数据 `D:\Study\26sp\parallel\ann\files\`。五份报告存于 `bench_results/windows_i9_13900h/`。

(1) Hotspots（flat）

函数	CPU Time	模块
malloc	0.033s	msvcrt.dll
`std::istream::read`	0.012s	libstdc++-6.dll

💡 热点 99% 集中在单个内循环函数，符合 ANN-Flat 的计算模式；I/O 开销可忽略。由于是 MinGW release 构建且未带调试符号，函数名以地址形式出现，通过对照 `main_win_avx2.exe` 反汇编可确认是 `flat_search`。

(2) Microarchitecture Exploration（flat / sq / pq）

指标	Flat-AVX2	SQ-AVX2 (p=100)	PQ-AVX2 (p=1000)
CPI (P-core)	0.723	**0.307**	**0.270**
Front-End Bound	2.9%	5.9%	8.0%
Back-End Bound	**59.0%**	23.0%	16.7%
Core Bound	24.4%	14.3%	16.1%
256-bit Int Vector (uOps)	0%	**28.6%**	0%
3+ Ports Utilized	17.4%	51.1%	**85.9%**
Avg CPU Frequency	⚠️ **1.484 GHz**（E-core 被调度）	3.094 GHz	3.109 GHz

✅ **Flat P-core 绑定重跑已完成**（用 PowerShell `Start-Process` + `ProcessorAffinity=0xFFFF`）：Avg Freq 从 **1.484 GHz → 5.086 GHz**（P-core Turbo Boost），CPI 0.723 → 0.904（P-core 纯粹场景下暴露出真实带宽瓶颈），256-bit FP Vector uOps 26.3% → 33.8%，DRAM Bandwidth 继续击中 91.9% 时钟。三次 flat latency 1470/1554/1700µs 与独立速度测量的 1756µs 一致。

🆕 P-core 绑定后的校正数据（同一 flat 负载）

指标	E-core混合（旧）	**P-core绑定（新）**	Avg CPU Frequency	1.484 GHz	**5.086 GHz**
CPI (P-core)	0.723	0.904	Retiring	19.5%	19.5%
Front-End Bound	2.9%	1.5%	Bad Speculation	18.6%	7.6%
Back-End Bound	59.0%	**71.4%**	Memory Bound	34.6%	**56.3%**
256-bit FP Vector uOps	26.3%	**33.8%**	FB Full	87.8%	55.8%
DRAM Bound	36.7%	49.1%	Memory Bandwidth (of clocks)	78.1%	**91.9%**
DRAM BW 峰值	41.8 GB/s	42.1 GB/s	LLC Miss	549.8M	448.4M

💡 **P-core 校正后的真实结论：**

- **Flat-AVX2 在 i9-13900H P-core 上是硬带宽瓶颈**——DRAM Bandwidth 占 91.9% 时钟，峰值 42 GB/s 接近平台 71 GB/s 极限的60%
- **CPI 0.904 不是 "算法不好"，而是计算能力大幅冗余于存储供给速率**——AVX2 256-bit FMA 每时钟 32 float，而 DRAM 跟不上
- **E-core 混合的 59% Back-End Bound 展下降为 P-core 独享的 71.4%**：P-core 更快而存储不更快，所以停滞占比反而上升
- **Machine Clears 从 18.6% 降到 7.3%**：确认了 E-core 调度的混乱会加剧 memory disambiguation nuke
- **Bandwidth bound 的实践假设**：未来加 pthread / OpenMP 在 flat 上的扩展性会显著打折，因为目前单线程已吃掉平台带宽 60%；SQ/PQ 无此问题，多线程扩展性会更好

💡 **微架构结论：**

- **Flat 是带宽瓶颈**：FB Full 87.8% + Back-End 59% + Memory Bound 34.6%，96 维 float32 全扫属典型 streaming 访问，被 L2→DRAM 路径拖住
- **SQ 兼顾计算与内存**：256-bit 整数 SIMD (`madd_epi16`) 占 28.6% uOps，Memory Bound 仅 8.7%，3+ ports 利用率 51%，是三种方法中平衡最好的
- **PQ retiring 率全场最高 68.5%**：ADC 查表 + 标量累加进入 micro-ops fusion 良好，端口利用 3+ 达 85.9%，但 elapsed 15.6s 的大头是 k-means 训练（build 阶段单线程 20 iter），查询阶段只占小头
- **Machine Clears 18.6% (flat) / 12.5% (sq)**：定位到 `priority_queue` 维护中 `if (dis < q.top().first)` 的 store-load 依赖触发 memory disambiguation nuke，与 Intel 优化手册 "Memory Disambiguation" 章节描述一致

(3) Memory Access（flat）

指标	值	P-core Memory Bound	41.9% of Pipeline Slots
P-core DRAM Bound	44.0% of Clockticks	P-core L3 Bound	7.3%
Loads / Stores	7.69B / 57.5M	LLC Miss Count	**549.8M**
DRAM Bandwidth 平均	**22.6 GB/s**	DRAM Bandwidth 峰值	**41.8 GB/s**（平台极限 69 GB/s）

📊 **带宽分析**：2000 queries × 37MB/query ≈ 74GB 的理论读入量，实测 DRAM 平均 22.6 GB/s × elapsed 15.6s ≈ 353 GB（受 cache reuse 影响），证明 **Flat-AVX2 在 i9-13900H 上已接近带宽上限**。这也是未来加 pthread 线性扩展性会被打折的主要原因——即使多个线程，DRAM 只能读一次数据。

(4) 跨平台 SIMD 宽度对比（补充到报告 3.3.4 节）

平台	SIMD	Flat 距离函数 uOps	说明	鲲鹏 920 / 平板	NEON (128-bit)	4× `float32x4_t` 累加器	每轮 16 f
i9-13900H	AVX2 (256-bit)	4× `__m256` 累加器 + FMA	每轮 32 float，单指令吞吐 2×	3090 Xeon 8358	AVX-512（未采集）	—	Day 14 可

🎯 当前任务与进度

实验目标

并行程序设计 Lab2 — SIMD 编程实验，选题 ANN（近似最近邻搜索）

评分体系

- **基础要求**得分上限：满分的 90% = **13.5 分**
- **进阶要求**加分：10% = **1.5 分**

进度

- ☑ ~~Day 1：环境搭建 + 代码阅读 + baseline 记录~~
- ☑ ~~Day 2：Flat SIMD 实现（NEON 4路展开，3.19x 加速）~~
- ☑ ~~Day 3-4：SQ SIMD 实现（uint8 量化 + 两阶段检索，7.61x 加速）~~
- ☑ ~~Day 5-7：PQ SIMD 实现（PQ 编码 + ADC 查询，完整 tradeoff 曲线）~~
- ☑ ~~Day 8：汇编分析 + 编译器自动向量化对比 + 不同优化等级性能对比~~
- ☑ ~~Day 9：鲲鹏 920 服务器提交测试通过（PQ SIMD 全 p 值数据已采集）~~
- ☑ ~~Day 10：鲲鹏补齐 Baseline / Flat SIMD / SQ SIMD / PQ SIMD **四版本完整数据**~~
- ☑ ~~Day 11：开发环境迁移到原生 Windows 11，代码结构重整 + git 提交~~
- ☑ ~~Day 12：鲲鹏 `result_*.txt` 已整理到 `bench_results/kunpeng_server/`（baseline_flat.csv + sq_simd.csv + pq_simd.csv + RESULTS.md + cleanup + ann/README.md 重写）~~
- ☑ ~~**Day 13**：Windows i9-13900H AVX2 实现 + 完整 VTune profiling + P-core 绑定校正数据 + 3 份 CSV + RESULTS.md + CLOSEOUT.md，已 git commit `ef7472f` 并 push 到 master~~

☐ Day 14（可选）：3090 服务器上补 AVX-512 版本，做 SIMD 宽度对比（128-bit NEON vs 256-bit AVX2 vs 512-bit AVX-512）。没有的话报告从 128-bit NEON 跳到 256-bit AVX2，不影响主结论

编译命令

```
# 本地编译
g++ main.cc -o main -O2 -fopenmp -lpthread -std=c++11
```

```
# 运行
./main
```

提交注意事项

- 只能通过 `test.sh` 提交，不能直接运行程序
- 新增文件均为 `.h` 格式（header-only）
- 数据路径已统一为 `/anndata/`，**不再需要切换**
- 禁止修改 `test.sh`
- 本地原生 Windows 不能编译（`arm_neon.h` 是 ARM 专用），仅用于代码编辑和 git

📌 Day 13 后续优化点（来自 agent 反馈）

- **VTune Hotspots 函数名显示为 `func@0x140001b60`**：因为 MinGW release 未带 debug symbols，后续可用 `-g -O2` 单独构一份 profiling 版本得到干净符号
- **Machine Clears 来源：priority_queue 的 top-k 维护**，如果继续榨性能可换成固定数组 / 分块 selection 结构
- **PQ VTune elapsed 包含 k-means build**，若要纯搜索段 profile，可加索引序列化或预热后只 profile search
- **Agent 发现的不一致**：
  - 3 份 P-core VTune txt 报告一开始缺失（已从 result dir 补导出）
  - `.gitignore` 在仓库根目录而非 `ann/.gitignore`（已加 `ann/main_win_avx2.exe`）
  - 额外的 `run_vtune_flat_pcore.sh` 未在 git add 清单里，未提交
  - 旧 `speed_*.txt` / `speed_summary.csv` 保留但以新 CSV + RESULTS.md 为准

📌 关键技术笔记

- 96 维是 4 的倍数，NEON `float32x4_t` 无需处理尾部
- 平板支持 `asimddp`（dot product），可用 `vdotq_u32` 加速 uint8 距离计算
- 新增的 SIMD 代码放在 `.h` 文件中（header-only 要求）
- 详细实验方案见 ⚡ ANNS SIMD 实验执行方案（Lab2）

💻 Flat-SIMD 代码（flat_scan_simd.h）

💻 SQ-SIMD 代码（sq_scan_simd.h）

💻 PQ-SIMD 代码（pq_scan_simd.h）

📝 ANNS SIMD 实验报告初稿

📄 鲲鹏服务器 — 四个版本的 main.cc

📖 README.md（GitHub 仓库）

⚡ PQ-SIMD LUT 代码（pq_scan_simd_lut.h）— 冲 14 分

🚀 PQ-FastScan 代码（pq_scan_fastscan.h）— 冲 15 分

📜 新增 main 文件：main_pqsimd_lut.cc / main_pqfastscan.cc